

**ANALISIS KECERDASAN BUATAN
SISTEM PENCARIAN RUTE TERPENDEK
UNIVERSITAS BENGKULU BERBASIS WEB
MENGUNAKAN ALGORITMA DIJKSTRA**



Disusun Oleh:

Nama : Nayla Viona Azahra

NPM : G1A024045

Kelas : A

Dosen Pengampu :

Ir. Arie Vatesia, S. T., M. T. I., Ph.D., IPP

**PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNIK
UNIVERSITAS BENGKULU**

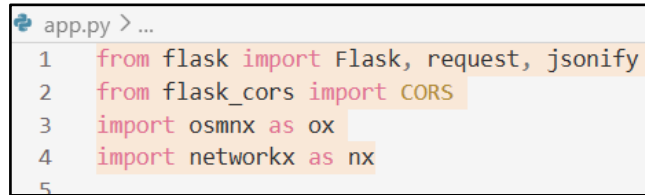
2026

ANALISIS SOURCE CODE

1. Import Library

Pada bagian awal program terdapat beberapa library yang digunakan untuk mendukung proses pengolahan data, perhitungan rute, dan komunikasi antara frontend dengan backend.

Printscreen Source Code:



```
app.py > ...
1 from flask import Flask, request, jsonify
2 from flask_cors import CORS
3 import osmnx as ox
4 import networkx as nx
5
```

Gambar 1 Source Code

Source Code:

```
from flask import Flask, request, jsonify
from flask_cors import CORS
import osmnx as ox
import networkx as nx
```

Penjelasan Source code:

Library **Flask** digunakan sebagai framework backend yang berfungsi untuk membuat web server dan menyediakan API yang dapat diakses oleh frontend. Objek request digunakan untuk menerima data yang dikirim pengguna, sedangkan jsonify digunakan untuk mengirimkan hasil perhitungan dalam format JSON. Library **Flask-CORS** digunakan untuk mengatasi masalah Cross-Origin Resource Sharing (CORS), sehingga frontend yang berjalan pada domain atau port berbeda tetap dapat berkomunikasi dengan backend. Library **OSMnx** digunakan untuk mengambil data jaringan jalan dari OpenStreetMap serta mengubahnya menjadi struktur graf yang dapat diproses lebih lanjut. Library **NetworkX** digunakan untuk melakukan analisis graf dan menjalankan algoritma pencarian rute terpendek.

2. Inisialisasi Aplikasi Flask

Setelah library diimpor, program membuat objek aplikasi Flask dan mengaktifkan fitur CORS.

Printscreen Source Code:

```

6  app = Flask(__name__)
7  CORS(app)

```

Gambar 2 Source Code

Source Code:

```

app = Flask(__name__)
CORS(app)

```

Penjelasan Source code:

Perintah `Flask(__name__)` digunakan untuk membuat instance aplikasi web Flask. Sedangkan fungsi `CORS(app)` mengizinkan akses API dari frontend yang mungkin berjalan pada server atau port yang berbeda dengan backend.

3. Pengambilan Data Jaringan Jalan

Program kemudian mengambil data jaringan jalan di sekitar Universitas Bengkulu menggunakan OSMnx.

Printscreen Source Code:

```

11  # Koordinat tengah Universitas Bengkulu
12  center_point = (-3.757, 102.272)
13
14  graph = ox.graph_from_point(
15      center_point,
16      dist=2000,
17      network_type="all"
18  )

```

Gambar 3 Source Code

Source Code:

```

center_point = (-3.757, 102.272)
graph = ox.graph_from_point(
    center_point,
    dist=2000,
    network_type="all"
)

```

Penjelasan Source code:

Variabel `center_point` berisi koordinat pusat Universitas Bengkulu yang digunakan sebagai titik referensi pengambilan data. Fungsi `graph_from_point()` digunakan untuk mengunduh data jalan dari OpenStreetMap dalam radius 2000

meter dari titik pusat tersebut.

Parameter:

- center_point: titik pusat area pengambilan data.
- dist=2000: radius pencarian sejauh 2 km.
- network_type="all": mengambil seluruh jenis jalur yang tersedia seperti jalan kendaraan, jalan kaki, dan jalur lainnya.

Hasil dari proses ini berupa graf jaringan jalan yang terdiri dari node (simpul) dan edge (sisi).

4. Membersihkan Graf

Setelah data diperoleh, program hanya mengambil komponen graf terbesar.

Printscreen Source Code:

```
20 graph = ox.truncate.largest_component(graph)
21 print("Graph jalan UNIB berhasil dimuat.")
22
```

Gambar 4 Source Code

Source Code:

```
graph = ox.truncate.largest_component(graph)
```

Penjelasan Source code:

Dalam data jaringan jalan sering terdapat bagian graf yang tidak terhubung dengan jaringan utama. Fungsi `largest_component()` digunakan untuk memilih komponen jaringan terbesar sehingga proses pencarian rute menjadi lebih stabil dan menghindari kesalahan ketika mencari jalur. Dengan langkah ini, sistem hanya menggunakan jaringan jalan yang benar-benar saling terhubung.

5. Halaman Utama Backend

Program menyediakan endpoint utama yang digunakan sebagai halaman awal backend.

Printscreen Source Code:

```
26 @app.route('/')
27 def home():
28     return "Backend Shortest Path UNIB Multi-Moda Berjalan"
29
```

Gambar 5 Source Code

Source Code:

```
@app.route('/')
def home():
```

```
return "Backend Shortest Path UNIB Multi-Moda Berjalan"
```

Penjelasan Source code:

Decorator `@app.route('/')` digunakan untuk menentukan URL utama aplikasi. Ketika pengguna mengakses alamat backend melalui browser, sistem akan menampilkan pesan: Backend Shortest Path UNIB Multi-Moda Berjalan. Bagian ini digunakan sebagai indikator bahwa server Flask berhasil dijalankan.

6. API Perhitungan Rute

Fitur utama program berada pada endpoint `/route`.

Printscreen Source Code:

```
33 @app.route('/route', methods=['POST'])
34 def route():
```

Gambar 6 Source Code

Source Code:

```
@app.route('/route', methods=['POST'])
def route():
```

Penjelasan Source code:

Endpoint ini digunakan untuk menerima data lokasi awal dan tujuan dari frontend. Metode **POST** dipilih karena data dikirim dalam bentuk JSON. Ketika pengguna memilih lokasi pada peta, frontend akan mengirimkan koordinat ke endpoint ini untuk diproses.

7. Validasi Data Masukan

Program melakukan pengecekan apakah data yang diterima memiliki format JSON.

Printscreen Source Code:

```
try:
    if not request.is_json:
        return jsonify({"error": "Request harus JSON"}), 400
```

Gambar 7 Source Code

Source Code:

```
if not request.is_json:
    return jsonify({"error": "Request harus JSON"}), 400
```

Penjelasan Source code:

Validasi ini penting untuk memastikan bahwa backend menerima data

dengan format yang benar. Jika format tidak sesuai, sistem akan mengembalikan pesan kesalahan dengan kode HTTP 400 (Bad Request).

8. Pengambilan Koordinat Pengguna

Data lokasi yang dikirim frontend kemudian diambil dan dikonversi menjadi tipe numerik.

Printscreen Source Code:

```
start_lat = float(data['startLat'])
start_lng = float(data['startLng'])
end_lat = float(data['endLat'])
end_lng = float(data['endLng'])
```

Gambar 8 Source Code

Source Code:

```
start_lat = float(data['startLat'])
start_lng = float(data['startLng'])
end_lat = float(data['endLat'])
end_lng = float(data['endLng'])
```

Penjelasan Source code:

Variabel tersebut menyimpan:

1. Latitude titik awal
2. Longitude titik awal
3. Latitude titik tujuan
4. Longitude titik tujuan

Koordinat ini nantinya digunakan untuk mencari node terdekat pada graf jalan.

9. Pemilihan Moda Transportasi

Program mendukung beberapa jenis moda transportasi.

Printscreen Source Code:

```
origin_node = ox.distance.nearest_nodes(graph, start_lng, start_lat)
destination_node = ox.distance.nearest_nodes(graph, end_lng, end_lat)
```

Gambar 9 Source Code

Source Code:

```
mode = data.get('mode', 'walk')
if mode == "car":
```

```

    speed_kmh = 25
elif mode == "bike":
    speed_kmh = 15
else:
    speed_kmh = 5

```

Penjelasan Source code:

Sistem menyediakan tiga pilihan moda:

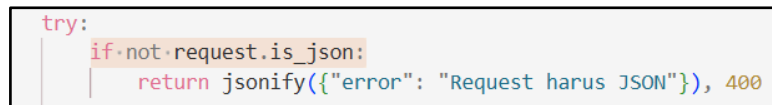
1. Jalan kaki (5 km/jam)
2. Sepeda (15 km/jam)
3. Kendaraan (25 km/jam)

Nilai kecepatan digunakan untuk menghitung estimasi waktu tempuh setelah rute ditemukan.).

10. Implementasi Algoritma Dijkstra

Setelah node asal dan tujuan ditemukan, sistem menghitung jalur terpendek.

Printscreen Source Code:



```

try:
    if not request.is_json:
        return jsonify({"error": "Request harus JSON"}), 400

```

Gambar 10 Source Code

Source Code:

```

shortest_route = nx.shortest_path(
    graph,
    origin_node,
    destination_node,
    weight='length'
)

```

Penjelasan Source code:

Bagian ini merupakan inti dari sistem.

Fungsi `shortest_path()` dari NetworkX menggunakan algoritma Dijkstra untuk mencari lintasan dengan bobot minimum.

Parameter:

1. `origin_node`: titik awal
2. `destination_node`: titik tujuan

3. `weight='length'`: bobot berdasarkan panjang jalan

Hasilnya berupa daftar node yang membentuk jalur terpendek.

11. Konversi Jalur Menjadi Koordinat

Jalur yang diperoleh kemudian diubah menjadi koordinat geografis.

Printscreen Source Code:

```
67 route_coords = [  
68     [graph.nodes[node]['y'], graph.nodes[node]['x']]  
69     for node in shortest_route
```

Gambar 11 Source Code

Source Code:

```
route_coords = [  
    [graph.nodes[node]['y'],  
     graph.nodes[node]['x']]  
    for node in shortest_route  
]
```

Penjelasan Source code:

Node pada graf hanya berupa ID. Agar dapat ditampilkan pada peta Leaflet, setiap node harus diubah menjadi koordinat latitude dan longitude. Koordinat inilah yang nantinya digambar sebagai garis rute pada peta.

12. Perhitungan Jarak Tempuh

Program menghitung total panjang jalur.

Printscreen Source Code:

```
72 total_distance = sum(  
73     graph[u][v][0]['length']  
74     for u, v in zip(shortest_route[:-1], shortest_route[1:])  
75 )
```

Gambar 12 Source Code

Source Code:

```
total_distance = sum(  
    graph[u][v][0]['length']  
    for u, v in zip(  
        shortest_route[:-1],  
        shortest_route[1:]  
    )  
)
```

Penjelasan Source code:

Setiap ruas jalan memiliki atribut **length** yang menunjukkan panjang jalan dalam meter. Program menjumlahkan seluruh panjang ruas yang dilalui sehingga diperoleh total jarak perjalanan.

13. Perhitungan Waktu Tempuh

Setelah jarak diketahui, sistem menghitung estimasi waktu perjalanan.

Printscreen Source Code:

```
waktu_menit = ((total_distance / 1000) / speed_kmh) * 60
```

Gambar 13 Source Code

Source Code:

```
waktu_menit = (  
    (total_distance / 1000)  
    / speed_kmh  
) * 60
```

Penjelasan Source code:

Rumus yang digunakan:

$$t = \frac{s}{v}$$

Keterangan:

- t = waktu tempuh
- s = jarak (km)
- v = kecepatan (km/jam)

Hasil akhirnya dikonversi menjadi satuan menit agar lebih mudah dipahami pengguna.

14. Pengiriman Hasil ke Frontend

Hasil perhitungan dikirim kembali ke frontend.

Printscreen Source Code:

```
79  
80  
81  
82  
83  
return jsonify({  
    "route": route_coords,  
    "distance": round(total_distance, 2),  
    "time": round(waktu_menit, 2)  
})
```

Gambar 14 Source Code

Source Code:

```
return jsonify({  
    "route": route_coords,  
    "distance": round(total_distance, 2),  
    "time": round(waktu_menit, 2)  
})
```

Penjelasan Source code:

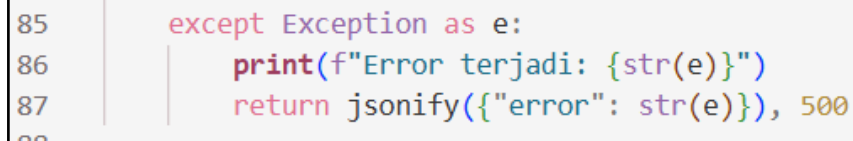
Backend mengirimkan:

1. koordinat jalur (route)
2. total jarak (distance)
3. estimasi waktu (time)

dalam format JSON sehingga dapat diproses dan ditampilkan oleh frontend.

15. Penanganan Kesalahan (Error Handling)

Program menggunakan blok try-except.

Printscreen Source Code:

```
85     except Exception as e:  
86         print(f"Error terjadi: {str(e)}")  
87         return jsonify({"error": str(e)}), 500  
88
```

Gambar 15 Source Code

Source Code:

```
except Exception as e:  
    return jsonify({"error": str(e)}), 500
```

Penjelasan Source code:

Bagian ini digunakan untuk mencegah program berhenti ketika terjadi kesalahan, misalnya koordinat tidak valid atau graf gagal diproses. Sistem akan mengirimkan pesan kesalahan kepada frontend sehingga pengguna mengetahui penyebab kegagalan proses.

ANALISIS PROPERTI GRAF

1. Analisis Properti Graf

Sistem pencarian rute terpendek Universitas Bengkulu memodelkan jaringan jalan sebagai sebuah graf berbobot (weighted graph). Dalam teori graf, graf merupakan struktur data yang terdiri dari sekumpulan simpul (*vertex/node*) dan sisi (*edge*) yang digunakan untuk merepresentasikan hubungan antar objek. Pada sistem ini, setiap lokasi atau persimpangan jalan direpresentasikan sebagai node, sedangkan jalan yang menghubungkan dua lokasi direpresentasikan sebagai edge.

Data graf diperoleh dari OpenStreetMap menggunakan library OSMnx. Data tersebut kemudian dikonversi menjadi graf yang dapat diproses oleh library NetworkX untuk melakukan pencarian jalur terpendek menggunakan algoritma Dijkstra. Dengan representasi graf ini, sistem dapat menentukan rute terbaik berdasarkan hubungan antar titik yang terdapat pada jaringan jalan kampus.

1.1 Node (Vertex)

Node merupakan titik-titik yang terdapat dalam jaringan jalan. Pada implementasi sistem, node dapat berupa persimpangan jalan, titik pertemuan beberapa ruas jalan, maupun lokasi tertentu yang terdapat pada area Universitas Bengkulu.

Sebagai contoh, jika terdapat jalur dari Gerbang Utama menuju Fakultas Teknik, maka beberapa persimpangan yang dilewati akan direpresentasikan sebagai node pada graf. Setiap node memiliki informasi koordinat geografis berupa latitude dan longitude yang digunakan untuk menampilkan posisi pada peta.

Contoh representasi node:

Node	Lokasi
N1	Gerbang Utama
N2	Persimpangan Jalan Utama
N3	Fakultas Teknik
N4	Fakultas Ekonomi

Node-node tersebut menjadi titik yang akan dilalui oleh algoritma Dijkstra dalam proses pencarian rute.

1.2 Edge (Sisi)

Edge merupakan penghubung antara dua node yang menunjukkan adanya jalur atau ruas jalan yang dapat dilalui. Setiap edge menyimpan informasi panjang jalan yang digunakan sebagai bobot dalam proses pencarian jalur.

Sebagai contoh:

Gerbang Utama ----- Persimpangan A ----- Fakultas Teknik

Pada ilustrasi tersebut terdapat dua edge, yaitu:

- Gerbang Utama → Persimpangan A
- Persimpangan A → Fakultas Teknik

Semakin banyak jaringan jalan yang tersedia, maka semakin banyak pula edge yang membentuk hubungan antar node dalam graf.

1.3 Graf Berbobot (Weighted Graph)

Graf yang digunakan dalam sistem termasuk kategori weighted graph karena setiap edge memiliki bobot berupa panjang jalan yang tersimpan pada atribut length.

Sebagai contoh:

Jalur	Bobot
A → B	100 meter
B → C	150 meter
A → C	300 meter

Jika pengguna ingin menuju titik C dari titik A, maka algoritma akan membandingkan seluruh kemungkinan jalur yang tersedia. Jalur:

A → B → C, memiliki total bobot: $100 + 150 = 250$ meter

sedangkan jalur langsung:

A → C, memiliki bobot: 300 meter

Karena 250 meter lebih kecil daripada 300 meter, maka algoritma Dijkstra akan memilih jalur melalui node B. Hal ini menunjukkan bahwa bobot pada setiap edge menjadi faktor utama dalam menentukan jalur terbaik.

1.4 Graf Terhubung (Connected Graph)

Graf yang digunakan pada sistem merupakan graf terhubung (*connected graph*). Hal ini karena program menggunakan fungsi:

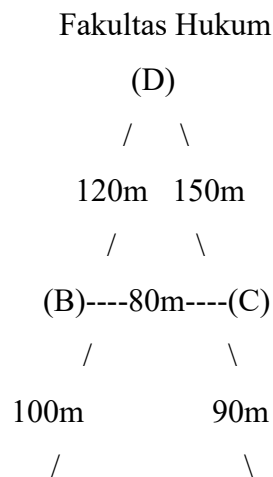
`graph = ox.truncate.largest_component(graph)`

Fungsi tersebut bertujuan mengambil komponen graf terbesar yang saling terhubung dan mengabaikan bagian jaringan yang terpisah.

Dengan kondisi graf yang terhubung, algoritma dapat menemukan jalur dari node asal menuju node tujuan tanpa mengalami kegagalan akibat tidak adanya hubungan antar node. Karakteristik ini sangat penting dalam sistem navigasi karena pengguna harus dapat berpindah dari satu lokasi ke lokasi lainnya melalui jaringan jalan yang tersedia.

1.5 Representasi Graf pada Sistem

Secara sederhana, graf pada sistem dapat direpresentasikan sebagai berikut:



Pada graf tersebut terdapat beberapa kemungkinan jalur dari Gerbang Utama menuju Fakultas Teknik:

1. $A \rightarrow E = 200$ meter
2. $A \rightarrow B \rightarrow C \rightarrow E = 270$ meter
3. $A \rightarrow B \rightarrow D \rightarrow C \rightarrow E = 460$ meter

Karena jalur $A \rightarrow E$ memiliki bobot paling kecil, maka jalur tersebut akan dipilih oleh algoritma Dijkstra sebagai rute optimal.

1.6 Hubungan Properti Graf dengan Algoritma Dijkstra

Keberhasilan algoritma Dijkstra dalam menemukan jalur terpendek sangat bergantung pada properti graf yang digunakan. Node berfungsi sebagai titik pencarian, edge berfungsi sebagai penghubung antar lokasi, sedangkan bobot digunakan sebagai dasar perhitungan biaya perjalanan. Karena graf

bersifat terhubung dan seluruh bobot bernilai positif, algoritma Dijkstra dapat bekerja secara optimal untuk menentukan rute dengan total jarak minimum.

1.7 Kesimpulan Analisis Properti Graf

Berdasarkan analisis yang dilakukan, jaringan jalan Universitas Bengkulu dapat dimodelkan sebagai graf berbobot dan terhubung yang terdiri atas node sebagai representasi lokasi serta edge sebagai representasi ruas jalan. Setiap edge memiliki bobot berupa panjang jalan yang digunakan oleh algoritma Dijkstra untuk menentukan rute terpendek. Representasi graf ini memungkinkan sistem melakukan proses pencarian jalur secara efektif sehingga mampu menghasilkan rute optimal dari lokasi asal menuju lokasi tujuan.

KOMPLEKSITAS ALGORITMA

1. Kompleksitas Algoritma

Kompleksitas algoritma merupakan ukuran yang digunakan untuk mengetahui seberapa besar kebutuhan waktu dan sumber daya komputasi yang diperlukan suatu algoritma dalam menyelesaikan permasalahan. Pada sistem pencarian rute terpendek Universitas Bengkulu, algoritma yang digunakan adalah Dijkstra Algorithm, yaitu algoritma pencarian jalur terpendek pada graf berbobot positif.

Algoritma Dijkstra bekerja dengan cara mencari node yang memiliki jarak sementara terkecil dari titik awal, kemudian memperbarui jarak menuju node-node tetangganya hingga seluruh node yang diperlukan telah dievaluasi. Proses ini dilakukan secara berulang sampai ditemukan jalur optimal menuju node tujuan.

1.1 Kompleksitas Waktu (Time Complexity)

Kompleksitas waktu menunjukkan jumlah operasi yang harus dilakukan algoritma untuk menyelesaikan suatu permasalahan. Pada implementasi sederhana menggunakan matriks ketetanggaan (*adjacency matrix*), algoritma Dijkstra memiliki kompleksitas:

$$O(V^2)$$

dengan:

$$V = \text{jumlah node (vertex)}$$

Namun, library NetworkX yang digunakan dalam sistem memanfaatkan struktur data yang lebih efisien berupa priority queue (heap) sehingga kompleksitas waktunya menjadi:

$$O((V + E) \log V)$$

Keterangan:

V = jumlah node

E = jumlah edge

Kompleksitas tersebut menunjukkan bahwa waktu eksekusi dipengaruhi oleh jumlah simpul dan jumlah ruas jalan yang terdapat pada graf. Semakin besar ukuran graf, semakin banyak proses yang harus dilakukan untuk menentukan jalur terpendek.

Sebagai ilustrasi, misalkan graf jaringan jalan memiliki:

1. node
2. 2.500 edge

Maka algoritma tidak perlu memeriksa seluruh kemungkinan jalur secara brute force, melainkan cukup melakukan eksplorasi berdasarkan node dengan jarak minimum sehingga proses pencarian tetap efisien.

1.2 Kompleksitas Ruang (Space Complexity)

Selain kompleksitas waktu, algoritma juga memerlukan memori untuk menyimpan informasi selama proses pencarian berlangsung. Pada algoritma Dijkstra, memori digunakan untuk menyimpan:

1. Data graf (node dan edge).
2. Daftar jarak sementara setiap node.
3. Priority queue.
4. Informasi node yang telah dikunjungi.

Kompleksitas ruang algoritma Dijkstra dapat dituliskan sebagai:

$$O(V + E)$$

Artinya kebutuhan memori akan bertambah seiring bertambahnya jumlah node dan edge pada graf. Namun karena area yang digunakan hanya mencakup lingkungan Universitas Bengkulu, kebutuhan memori masih tergolong kecil dan dapat ditangani dengan baik oleh komputer maupun laptop standar.

1.3 Pengaruh Ukuran Graf terhadap Performa

Performa algoritma dipengaruhi oleh jumlah node dan edge yang terdapat dalam jaringan jalan. Semakin luas area yang dipetakan, semakin besar pula ukuran graf yang harus diproses.

Sebagai contoh:

Kondisi Graf	Jumlah Node	Jumlah Edge	Performa
Area kecil	Sedikit	Sedikit	Sangat cepat
Area kampus	Sedang	Sedang	Cepat
Area kota	Banyak	Banyak	Lebih lambat

Pada sistem yang dikembangkan, area pencarian hanya mencakup lingkungan Universitas Bengkulu sehingga jumlah node dan edge relatif tidak terlalu besar. Oleh karena itu, proses pencarian rute dapat dilakukan dalam waktu

yang sangat singkat dan tidak menimbulkan keterlambatan yang berarti bagi pengguna.

1.4 Efisiensi Algoritma Dijkstra

Algoritma Dijkstra dipilih karena memiliki tingkat efisiensi yang baik untuk graf berbobot positif seperti jaringan jalan. Setiap ruas jalan memiliki panjang yang bernilai positif sehingga memenuhi syarat penggunaan algoritma Dijkstra.

Selain itu, algoritma ini memiliki keunggulan berupa:

1. Mampu menghasilkan jalur optimal (optimal solution).
2. Stabil untuk berbagai ukuran graf.
3. Cocok digunakan pada sistem navigasi dan pencarian rute.
4. Didukung langsung oleh library NetworkX sehingga mudah diimplementasikan.

Namun demikian, Dijkstra juga memiliki keterbatasan, yaitu tidak menggunakan informasi heuristik dalam proses pencarian. Pada graf yang sangat besar, algoritma lain seperti A^* (*A-Star*) dapat memberikan performa yang lebih cepat karena memanfaatkan estimasi jarak menuju tujuan.

1.5 Kesimpulan Kompleksitas Algoritma

Berdasarkan analisis yang dilakukan, algoritma Dijkstra yang digunakan pada sistem memiliki kompleksitas waktu sebesar $O((V+E) \log V)$ dan kompleksitas ruang sebesar $O(V+E)$. Kompleksitas tersebut tergolong efisien untuk digunakan pada jaringan jalan Universitas Bengkulu yang memiliki ukuran graf relatif sedang. Dengan karakteristik graf yang berbobot positif dan terhubung, algoritma Dijkstra mampu menemukan jalur terpendek secara optimal dengan waktu eksekusi yang cepat sehingga sesuai untuk diterapkan pada sistem navigasi berbasis web.

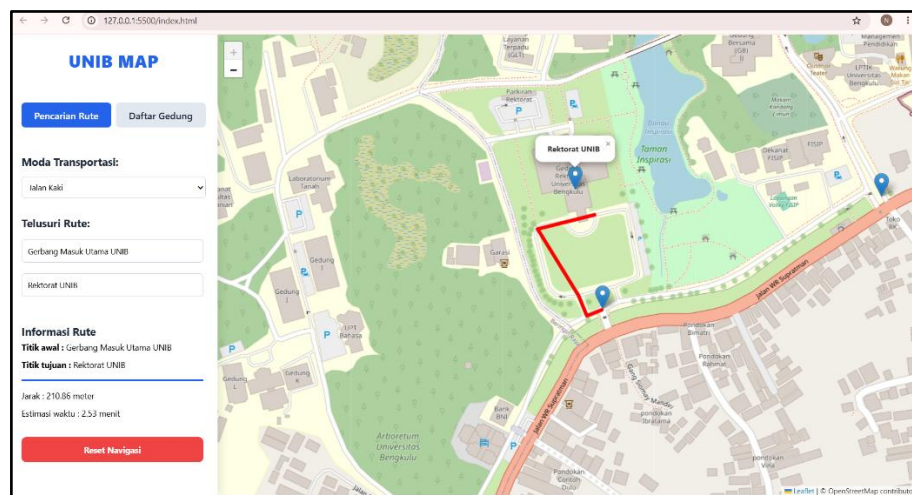
ANALISIS SKENARIO PENGUJIAN

Analisis skenario pengujian dilakukan untuk mengetahui bagaimana algoritma Dijkstra bekerja dalam berbagai kondisi pencarian rute pada lingkungan Universitas Bengkulu. Pengujian ini bertujuan untuk mengevaluasi kemampuan algoritma dalam menemukan jalur terpendek, menyesuaikan diri terhadap perubahan tujuan, serta memilih rute terbaik ketika terdapat beberapa alternatif jalur yang tersedia. Hasil pengujian dianalisis berdasarkan prinsip kerja algoritma dan karakteristik graf yang digunakan.

1. Skenario 1: Pencarian Rute dengan Jarak Dekat

Pada skenario pertama, pengujian dilakukan dengan memilih lokasi awal dan tujuan yang berada pada area yang relatif berdekatan, misalnya dari Gerbang Masuk Utama UNIB menuju Gedung Rektorat. Pada kondisi ini jumlah node dan edge yang harus dieksplorasi oleh algoritma relatif sedikit karena jarak antar lokasi tidak terlalu jauh.

Ketika pengguna memilih kedua lokasi tersebut, sistem akan mencari node terdekat dari titik awal dan tujuan, kemudian menjalankan algoritma Dijkstra untuk menentukan jalur dengan total bobot terkecil. Karena ruang pencarian yang relatif kecil, algoritma dapat menemukan solusi dengan cepat.



Gambar 1 Skenario 1

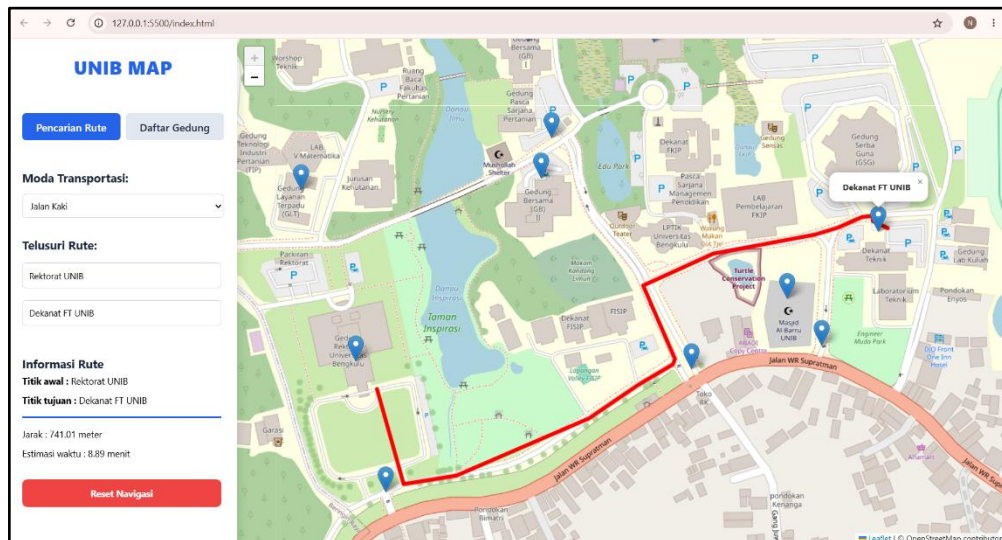
Hasil pengujian menunjukkan bahwa sistem mampu menghasilkan rute secara hampir instan dengan jalur yang sesuai dengan kondisi jaringan jalan yang tersedia. Jarak tempuh yang dihasilkan juga sesuai dengan panjang jalan yang terdapat pada data OpenStreetMap.

Dari hasil tersebut dapat disimpulkan bahwa algoritma Dijkstra bekerja sangat efisien pada kasus pencarian jarak dekat karena jumlah node yang dievaluasi relatif sedikit.

2. Skenario 2: Pencarian Rute dengan Jarak Jauh

Pada skenario kedua, lokasi awal dan tujuan dipilih pada area yang berjauhan, misalnya dari Gedung Layanan Terpadu menuju FKIK UNIB.

Dalam kondisi ini, jumlah node yang harus dievaluasi menjadi lebih banyak dibandingkan skenario pertama karena algoritma harus mempertimbangkan lebih banyak ruas jalan dan persimpangan yang mungkin dilalui. Meskipun demikian, algoritma Dijkstra tetap melakukan proses pencarian secara sistematis dengan selalu memilih node yang memiliki jarak minimum sementara.



Gambar 2 Skenario 2

Hasil pengujian menunjukkan bahwa sistem tetap mampu menghasilkan rute optimal dengan baik meskipun jalur yang dicari lebih panjang. Waktu pemrosesan sedikit meningkat dibandingkan pencarian jarak dekat, tetapi perbedaannya tidak terlalu signifikan karena ukuran graf kampus masih relatif kecil.

Pengujian ini menunjukkan bahwa algoritma Dijkstra mampu mempertahankan kinerjanya dalam mencari jalur terpendek meskipun jumlah node yang harus diproses bertambah.

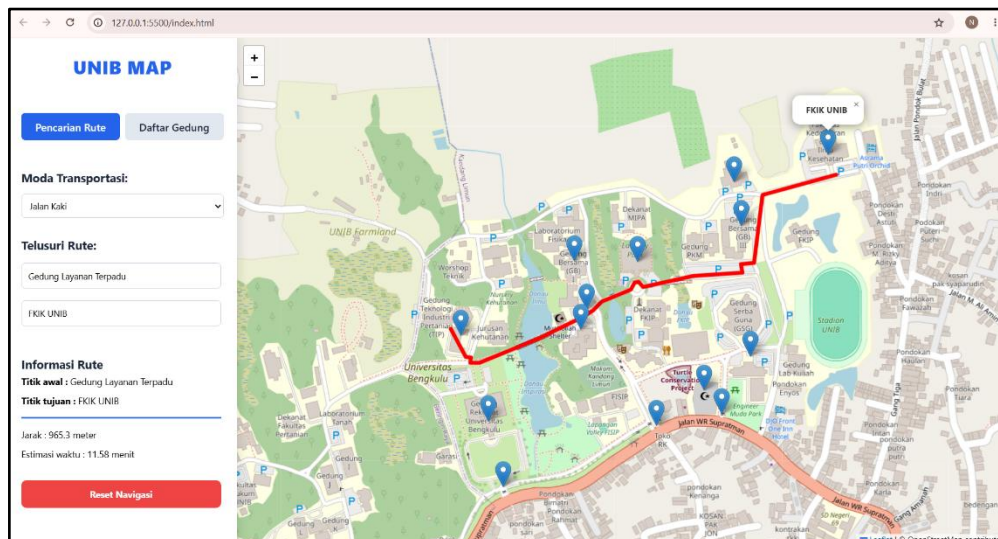
3. Skenario 3: Terdapat Beberapa Alternatif Jalur

Pada jaringan jalan kampus sering ditemukan lebih dari satu jalur yang dapat digunakan untuk mencapai tujuan yang sama. Oleh karena itu, dilakukan pengujian pada kondisi dimana terdapat beberapa alternatif rute antara lokasi awal dan tujuan.

Sebagai ilustrasi, misalkan terdapat tiga kemungkinan jalur:

Jalur	Total Jarak
A → B → D	500 meter
A → C → D	420 meter
A → E → D	650 meter

Pada kondisi tersebut, algoritma Dijkstra akan menghitung total bobot dari setiap kemungkinan jalur yang tersedia. Setelah seluruh perhitungan dilakukan, algoritma akan memilih jalur yang memiliki total bobot paling kecil.



Gambar 3 Skenario 3

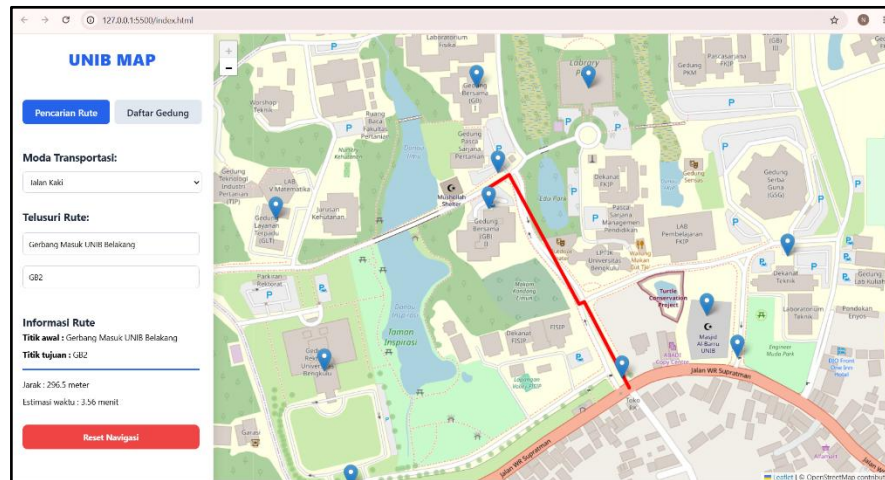
Hasil pengujian menunjukkan bahwa sistem memilih jalur yang memiliki total jarak paling pendek dibandingkan alternatif lainnya. Hal ini membuktikan bahwa algoritma Dijkstra berhasil menemukan solusi optimal sesuai dengan prinsip kerja shortest path.

Skenario ini menjadi salah satu pengujian penting karena menunjukkan kemampuan algoritma dalam menentukan keputusan terbaik ketika tersedia banyak pilihan jalur.

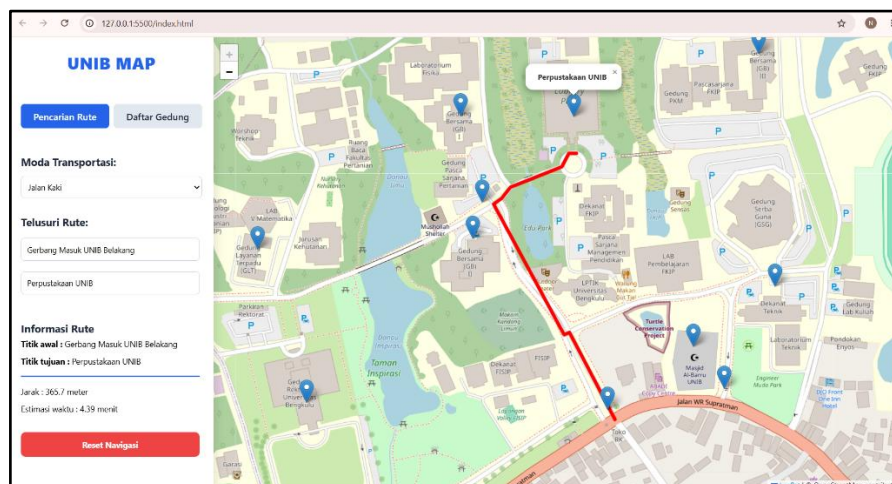
4. Skenario 4: Perubahan Titik Tujuan

Pengujian selanjutnya dilakukan dengan mengubah lokasi tujuan secara berulang sementara titik awal tetap sama. Tujuannya adalah untuk melihat apakah sistem mampu menghasilkan rute yang berbeda sesuai dengan perubahan tujuan yang diberikan pengguna.

Pada pengujian ini, pengguna memilih satu titik awal kemudian mencoba beberapa lokasi tujuan yang berbeda. Setiap kali tujuan diubah, sistem akan menjalankan kembali algoritma Dijkstra untuk mencari jalur terpendek menuju lokasi baru.



Gambar 4 Skenario 4 pertama



Gambar 5 Skenario 4 kedua

Hasil pengujian menunjukkan bahwa rute yang ditampilkan selalu berubah menyesuaikan posisi tujuan yang dipilih. Selain itu, jarak dan estimasi waktu tempuh yang dihasilkan juga berubah sesuai dengan panjang jalur yang ditemukan.

Pengujian ini membuktikan bahwa sistem mampu merespons perubahan input pengguna dengan baik dan tetap menghasilkan solusi yang valid untuk setiap tujuan yang berbeda.

5. Evaluasi Hasil Pengujian

Berdasarkan seluruh skenario yang telah dilakukan, algoritma Dijkstra mampu bekerja dengan baik pada berbagai kondisi pencarian rute. Algoritma berhasil menemukan jalur terpendek pada pencarian jarak dekat maupun jarak jauh, mampu memilih jalur terbaik ketika terdapat beberapa alternatif rute, serta dapat menghasilkan solusi yang berbeda sesuai perubahan lokasi tujuan.

Selain menghasilkan jalur yang optimal, waktu pemrosesan yang dibutuhkan juga relatif cepat karena ukuran graf jaringan jalan Universitas Bengkulu tidak terlalu besar. Dengan demikian, algoritma Dijkstra dinilai cukup efektif dan sesuai untuk diterapkan pada sistem pencarian rute terpendek berbasis web yang dikembangkan dalam penelitian ini.

EVALUASI ALGORITMA

Evaluasi algoritma dilakukan untuk menilai tingkat keberhasilan algoritma Dijkstra dalam menyelesaikan permasalahan pencarian rute terpendek pada lingkungan Universitas Bengkulu. Evaluasi ini mencakup kemampuan algoritma dalam menemukan solusi optimal, efisiensi proses pencarian, serta kesesuaiannya dengan karakteristik data yang digunakan pada sistem.

Berdasarkan hasil implementasi dan pengujian yang telah dilakukan, algoritma Dijkstra mampu menemukan jalur dengan total jarak minimum dari lokasi awal menuju lokasi tujuan. Pada setiap skenario pengujian, algoritma selalu menghasilkan rute yang valid dan konsisten sesuai dengan bobot jalan yang terdapat pada graf. Hal ini menunjukkan bahwa algoritma Dijkstra memiliki tingkat akurasi yang baik dalam menyelesaikan permasalahan shortest path.

Dari sisi performa, waktu pemrosesan yang dibutuhkan relatif cepat karena ukuran graf yang digunakan hanya mencakup area Universitas Bengkulu. Meskipun jumlah node dan edge cukup banyak, algoritma tetap mampu menghasilkan rute dalam waktu singkat sehingga pengguna tidak merasakan keterlambatan saat menggunakan sistem. Dengan kompleksitas waktu sebesar $O((V+E) \log V)$, algoritma masih sangat layak digunakan untuk kebutuhan navigasi pada skala kampus.

Selain itu, algoritma Dijkstra juga memiliki keunggulan karena mampu memberikan solusi optimal tanpa memerlukan informasi tambahan berupa heuristik. Selama seluruh bobot pada graf bernilai positif, algoritma dapat menjamin bahwa jalur yang dihasilkan merupakan jalur terpendek. Karakteristik ini sangat sesuai dengan data jaringan jalan yang digunakan dalam sistem, dimana panjang jalan selalu memiliki nilai positif.

Namun demikian, algoritma Dijkstra memiliki beberapa keterbatasan. Algoritma harus mengevaluasi cukup banyak node ketika jarak antara titik awal dan tujuan semakin jauh. Pada graf yang berukuran sangat besar, proses pencarian dapat menjadi lebih lambat dibandingkan algoritma lain seperti A* (A-Star) yang memanfaatkan fungsi heuristik untuk mempercepat pencarian menuju tujuan.

Secara keseluruhan, algoritma Dijkstra dinilai sangat sesuai untuk diterapkan pada sistem pencarian rute terpendek Universitas Bengkulu karena mampu

menghasilkan solusi yang optimal, memiliki performa yang baik pada ukuran graf yang digunakan, serta mudah diimplementasikan menggunakan library NetworkX.

Tabel Evaluasi Algoritma

Aspek Evaluasi	Hasil
Keakuratan Jalur	Sangat Baik
Kemampuan Menemukan Solusi Optimal	Baik
Kecepatan Proses	Baik
Stabilitas Hasil	Baik
Kemudahan Implementasi	Sangat Baik
Kesesuaian dengan Graf Berbobot Positif	Sangat Baik

KESIMPULAN ANALISIS

Berdasarkan analisis graf, kompleksitas algoritma, serta berbagai skenario pengujian yang telah dilakukan, dapat disimpulkan bahwa algoritma Dijkstra berhasil diterapkan pada sistem pencarian rute terpendek Universitas Bengkulu berbasis web. Jaringan jalan kampus dapat direpresentasikan dalam bentuk graf berbobot yang terdiri dari node sebagai lokasi dan edge sebagai penghubung antar lokasi.

Hasil pengujian menunjukkan bahwa algoritma mampu menemukan jalur terpendek secara konsisten pada berbagai kondisi, baik untuk jarak dekat, jarak jauh, maupun ketika terdapat beberapa alternatif jalur yang tersedia. Selain itu, sistem juga mampu menyesuaikan hasil pencarian ketika pengguna mengubah lokasi tujuan.

Dari sisi performa, algoritma Dijkstra memiliki kompleksitas waktu $O((V+E) \log V)$ yang masih efisien untuk ukuran graf jaringan jalan kampus. Dengan karakteristik graf yang berbobot positif dan saling terhubung, algoritma dapat bekerja secara optimal dan menghasilkan solusi yang akurat.

Oleh karena itu, algoritma Dijkstra dapat dinyatakan sebagai metode yang tepat untuk menyelesaikan permasalahan pencarian rute terpendek pada sistem navigasi Universitas Bengkulu. Implementasi algoritma ini tidak hanya berhasil memberikan solusi yang optimal, tetapi juga menunjukkan penerapan konsep kecerdasan buatan dan teori graf dalam menyelesaikan permasalahan nyata di bidang navigasi dan sistem informasi geografis.

